

Fiori Elements 開發課程 4：Custom Page

環境：BTP ABAP Environment Trial (沿用 fe01 的 `fe01_connection_test` 專案)

Lecture

這一課要解決的問題

Note 的 `content` 是長文字欄位，fe01 ~ fe03 一路用的標準 Object Page 範本，把它當成一般短欄位塞進表單裡顯示——一行文字、跟 Note ID / Last Changed 擠在同一排，長內容完全沒有適合的排版空間。這正是官方文件對 Custom Page 的定位：「超出範本能表達的畫面」，範本的通用表單邏輯沒辦法幫你決定「這個欄位該用大面積的文字區塊呈現」這種客製化排版決策。

什麼是 Custom Page (先查證官方文件，不是憑印象寫)

依據官方 `SAP-docs/sapui5` 文件 (`extension-points-for-pages`)：Custom Page 是 Fiori Elements Flexible Programming Model 的延伸機制——在 `manifest.json` 的 `routing.targets` 裡，把某個頁面的 Target 設成 `type: "Component"`、`name: "sap.fe.core.fpm"`，用 `viewName` 指向你自己寫的 XML View，`contextPath` 指定綁定的 Entity。跟 List Report / Object Page 「完全交給框架範本生成畫面」不同，Custom Page 是「你自己刻 View，但 OData V4 的 Binding Context 還是框架自動幫你準備好，View 裡可以直接綁欄位，也還能用 Fiori Elements 的 Building Block (如 `macros:Table`)」——介於「全自動範本」跟「完全自己刻的 Freestyle UI5 App」之間。

查證過程：Page Map 的 + 按鈕，走不通

VS Code 裡有個 **Page Map** (Command Palette 打 `Fiori: Show Page Map`，⚠️ 不是 `Open Page Map`——這是實際指令名稱，跟直覺猜的不一樣)，可以用圖形化方式管理 App 的頁面結構。

⚠️ 第一次打開 **Page Map**，畫面完全空白 (沒有畫出任何頁面方塊)，VS Code 右下角還跳出一個錯誤通知：

```
EntryNotFound (FileSystemError): ffs:/.../fe01_connection_test/src/app.json
Source: SAP Fiori Tools - Application Modeler
```

Workaround：執行一次 **Developer: Reload Window**，重新打開 Page Map 後，`NoteList` (List Report) → `NoteObjectPage` (Object Page) 的頁面方塊跟導覽箭頭就正常畫出來了。這是這個工具版本的一個實際小毛病，第一次用 Page Map 遇到空白畫面，先試這招，不用懷疑是專案設定壞了。

每個頁面方塊右上角有 + (新增頁面) / 鉛筆 (開啟 Page Editor) / ↗ (Show Controller Extension) / 垃圾桶 (Delete Page) 四個圖示。點 `NoteObjectPage` 的 +，滑鼠移上去顯示 `No navigation targets available`，整個按鈕是反灰、點不動的。

原因：Page Map 的 + 是拿來新增「透過 OData Navigation Property 才能到達」的頁面 (例如點某個關聯 Entity 繼續往下鑽)。`ZI_RC05_NOTE` 是一個完全平坦的 Entity (rc05 / rc08 從沒幫它加過 Composition / Association)，沒有任何導覽關係可以延伸，所以這個入口對它完全用不上——這不是操作錯誤，是這個資料模型天生沒有「新增子頁面」這個選項。

另外查了鉛筆圖示 (**Page Editor**)：點開只能編輯「範本頁面本身的結構」(Header / Sections / Footer 這些既有範本元件怎麼排列，包括 fe03 加的 `General Information` / `Audit Information` 兩個 Section)，沒有找到任何「把這個頁面整個換成 Custom Page」的按鈕。

結論：這系統版本的 Page Map / Page Editor 沒有 GUI 精靈可以把一個既有的範本頁面「轉換」成 Custom Page，官方文件說的「手動改 `manifest.json` routing target」才是正規做法——這不是我漏找了什麼捷徑，是改用 **Page Map** 直接建立新頁面 (需要 Navigation Property) 之外，取代既有頁面這條路本來就沒有 GUI 工具支援。

實作：把 `NoteObjectPage` 換成 Custom Page

manifest.json 改動 (`routing.targets.NoteObjectPage`) :

```
// 改之前
"NoteObjectPage": {
  "type": "Component", "id": "NoteObjectPage",
  "name": "sap.fe.templates.ObjectPage",
  "options": { "settings": { "editableHeaderContent": false, "contextPath": "/Note" } }
}

// 改之後
"NoteObjectPage": {
  "type": "Component", "id": "NoteObjectPage",
  "name": "sap.fe.core.fpm",
  "options": { "settings": {
    "viewName": "fe01connectiontest.ext.view.NoteDetail",
    "contextPath": "/Note"
  } }
}
```

新增的 **View** (`webapp/ext/view/NoteDetail.view.xml`) :

```
<mvc:View
  controllerName="fe01connectiontest.ext.controller.NoteDetail"
  xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m" xmlns:core="sap.ui.core"
  xmlns:layout="sap.ui.layout">
  <Page showNavButton="true" navButtonPress=".onNavBack">
    <customHeader>
      <Bar><contentMiddle><Title text="{title}" level="H2"/></contentMiddle></Bar>
    </customHeader>
    <content>
      <layout:VerticalLayout class="sapUiContentPadding" width="100%">
        <ObjectAttribute title="Note ID" text="{note_id}"
class="sapUiSmallMarginBottom"/>
        <ObjectAttribute title="Last Changed" text="{changed_at}"
class="sapUiMediumMarginBottom"/>
        <Panel headerText="Content" width="100%" class="sapUiResponsiveMargin">
          <FormattedText htmlText="{content}" class="sapUiSmallMargin"/>
        </Panel>
      </layout:VerticalLayout>
    </content>
  </Page>
</mvc:View>
```

新增的 **Controller** (`webapp/ext/controller/NoteDetail.controller.js`) ——官方文件明講 Custom Page 的 Controller 要繼承 `sap/fe/core/PageController` (不是普通的 `sap/ui/core/mvc/Controller`) · 這樣才能拿到 FE 框架提供的其他能力 (雖然這一課的例子很簡單、用不太到) :

```
sap.ui.define(["sap/fe/core/PageController"], function (PageController) {
  "use strict";
  return PageController.extend("fe01connectiontest.ext.controller.NoteDetail", {
    onNavBack: function () {
```

```

        window.history.back();
    }
});
});

```

檔案命名慣例：`viewName` 用 "`<sap.app.id>.ext.view.<檔名>`" 這種點記法（對應 `webapp/ext/view/<檔名>.view.xml` 這個實體路徑），`ext` 這個資料夾名稱是社群慣例（放「擴充」內容），不是官方強制規定，但沿用它可以讓其他開發者一看資料夾名稱就知道這是客製化內容，不是範本自動生成的。

實測結果與一個重要的取舍

livereload 自動刷新後，畫面完全符合預期：標題列顯示 `title`、`Note ID / Last Changed` 用 `ObjectAttribute` 各自一行呈現，`content` 放進獨立的 `Content Panel`、用 `FormattedText` 顯示——排版比原本範本的通用表單清楚很多。

⚠️⚠️ 但畫面上的 **Edit / Delete** 按鈕完全消失了——這是換成 Custom Page 必然的代價：範本頁面（`sap.fe.templates.ObjectPage`）的 Create / Edit / Delete / Draft 相關工具列，是範本自己生成的，換成 **Custom Page** 之後，這一整套 UI 沒有了，因為畫面完全是你自己刻的，框架不會偷偷幫你加回來。如果這一課要恢復刪除功能，得自己在 View 裡加 `Button`、Controller 裡自己寫 EML / 呼叫 OData Action 觸發刪除——這已經超出這一課的範圍，重點是先體會到這個取舍的存在。

⚠️ 額外發現：`showNavButton` / `navButtonPress` 這兩個屬性在這個 View 裡其實沒有作用——因為同時定義了 `<customHeader>`，`sap.m.Page` 的規則是一旦有 `customHeader`，標準 Header（含 `showNavButton` 產生的返回按鈕）整個被取代，`navButtonPress` 綁定的 `.onNavBack` 沒有對應的按鈕可以觸發。畫面左上角看到的 ◀ 返回箭頭，其實是 Fiori Elements App Shell 本身提供的導覽元件（List Report / Object Page 切換時框架自動維護的麵包屑），跟這個 View 自己的設定無關——這是寫 Custom Page 時很容易誤判的細節：寫了 `navButtonPress` 不代表它一定會被觸發，要看有沒有真的搭配一個會呼叫它的按鈕。

學習目標

- 能講出 Custom Page 的核心機制：`routing.targets` 的 `type: "Component" / name: "sap.fe.core.fpm" / viewName / contextPath`，以及它跟純 Freestyle UI5 App 的差異（Binding Context 跟 FE Building Block 還在，畫面自己刻）
- 知道 Page Map 的 + 新增頁面功能，前提是該頁面對應的 Entity 要有 OData Navigation Property——平坦 Entity（無 Composition / Association）沒有這個選項，不是操作錯誤
- 知道「取代既有範本頁面」沒有 GUI 精靈，只能手動改 `manifest.json`
- 能寫出 Custom Page 的 View（Controller 要繼承 `sap/fe/core/PageController`）跟對應的 `viewName` 命名慣例
- 知道換成 **Custom Page** 的代價：範本自動生成的 Create / Edit / Delete / Draft 工具列會完全消失，要自己重建
- 知道 `<customHeader>` 會取代 `sap.m.Page` 的標準 Header，連帶讓 `showNavButton` / `navButtonPress` 失效——寫 Custom Page 時這類「屬性設了但沒有實際效果」的細節要靠實測才會發現，不能只看語法有沒有錯

物件清單

延續 fe01 ~ fe03，繼續在同一個前端專案上疊加，沒有新增任何 ABAP 物件：

檔案	這一課的修改
<code>fe01_connection_test/webapp/manifest.json</code>	<code>routing.targets.NoteObjectPage</code> 從 <code>sap.fe.templates.ObjectPage</code> 換成 <code>sap.fe.core.fpm</code>
<code>fe01_connection_test/webapp/ext/view/NoteDetail.view.xml</code>	新增，Custom Page 的 View
<code>fe01_connection_test/webapp/ext/controller/NoteDetail.controller.js</code>	新增，繼承 <code>sap/fe/core/PageController</code>

動手練習

輪到你了：

1. 幫 `NoteDetail.view.xml` 加一個 `Button` (放在 `Page` 的 `footer` 裡)，按下去用 `MessageToast.show(...)` 顯示一段訊息——體驗一下在 `Custom Page` 裡加互動元件、寫 `Controller` 方法的完整流程 (跟寫一般 `UI5 View` 完全一樣，沒有任何 `FE` 特殊限制)
2. 想一想 (不用真的做)：如果要在這個 `Custom Page` 裡重新加回 `Delete` 功能，你會怎麼做？ (提示：查一下 `sap.fe.core.actions` 或 `EML` 在 `JS` 端的對應寫法，這是進階題，這一課不要求做出來)
3. 把 `<customHeader>` 拿掉，改回單純用 `title` 屬性 (`<Page title="{title}" showNavButton="true" navButtonPress=".onNavBack">`)，重新整理後確認這次 `showNavButton` 產生的返回按鈕有沒有出現、點下去會不會觸發 `onNavBack`——驗證這一課「`customHeader` 會蓋掉標準 `Header`」這個結論

驗證方式

這一課沒有 `ABAP` 物件變更，驗證方式是「畫面行為符合預期」：

1. 點進任一筆 `Note`，畫面變成自訂排版 (標題 + `Note ID` / `Last Changed` + 獨立的 `Content Panel`)，不再是原本的通用表單
2. 確認畫面上沒有 `Edit` / `Delete` 按鈕 (`Custom Page` 不會自動繼承範本的工具列)
3. 這一課實測結果：已截圖確認 `Custom Page` 正確顯示，排版跟預期一致，`Edit` / `Delete` 消失的取捨也已驗證屬實

⚠ 後續課程提醒 (2026-08-24 補記)

fe14 (訊息模型) 實測時踩到這個坑：fe14 要在 `Object Page` 測試 `Validation` 錯誤訊息，預設會用這一課換掉的 `Custom Page` (唯讀，沒有可輸入的 `Input` 欄位)，完全無法測試——如果之後要用 `fe01_connection_test` 測任何需要「真的能編輯欄位」的東西，記得先把 `manifest.json` 的 `NoteObjectPage` 切回本課最上面「改之前」的標準範本版本，測完再視需要切回來。這是「同一個專案被多堂課輪流疊加修改」的具體後遺症案例，見 fe14 講義開頭的完整說明。

思考題

1. 這一課的 `Custom Page` 完全取代了 `NoteObjectPage`——`List Report` 那邊完全沒有動過，還是原本的範本頁面。如果只想「加一個全新的、額外的頁面」(不取代任何既有頁面)，而且這個新頁面不需要透過 `Navigation Property` 到達 (例如一個獨立的統計/儀表板頁面)，你覺得 `manifest.json` 的 `routing.routes` / `routing.targets` 要怎麼設計？ (提示：想一想 fe02 學過的 `pattern` 語法，是不是可以自己定義一個全新的 `route`，不用透過 `Page Map` 的 `+`)
2. 這一課失去了 `Draft` 的 `Create` / `Edit` 按鈕——如果之後要在 `Custom Page` 裡重新做出「編輯模式」的體驗 (例如按一個按鈕切換成可編輯的欄位)，你覺得需要自己管理哪些狀態？ (提示：想一想 rc05 學過的 `Draft` 機制本身——`Activate` / `Discard` / `Edit` 這幾個 `Draft Action` 依然存在於後端 `BDEF`，`Custom Page` 只是沒有現成的按鈕幫你觸發它們而已)
3. `PageController` 這個基底類別，這一課完全沒有用到它提供的任何額外能力 (只用了最基本的 `extend`)。如果之後要在 `Custom Page` 裡讀取目前的 `Binding Context` (例如取得 `note_id` 做一些額外的邏輯判斷)，你會怎麼查證 `PageController` 有沒有提供對應的 `API`？ (提示：這是一個「不確定就去查證，不要憑印象亂猜」的練習——這一課多次強調外部文件可能跟實際工具行為有落差，遇到 `PageController` 的 `API` 細節，你會選擇查哪裡)

答案

見 `fe01_connection_test/webapp/manifest.json` (`NoteObjectPage` target)、
`fe01_connection_test/webapp/ext/view/NoteDetail.view.xml`、
`fe01_connection_test/webapp/ext/controller/NoteDetail.controller.js`。沒有新增或修改任何 `ABAP` 物件，也沒有新增前端專案——延續 fe01 ~ fe03 對同一個 `fe01_connection_test` 專案的疊加修改。實測結果：`Custom Page` 正確顯示，`Edit` / `Delete` 按鈕消失的取捨、`customHeader` 蓋掉 `showNavButton` 的細節，都已透過瀏覽器截圖驗證。